Database Management Systems Project

Library Management System

Review - 2

Team Members

L Srinath (24BDS0265)
Sugash K (24BCE2565)

Database Implementation and Core Functionalities

Objective

To implement the MongoDB design proposed in Review 1 and demonstrate database creation, representative data, CRUD operations, indexing, aggregation, validation, authentication and transaction-based circulation workflows through a working application.

Implementation summary

Item	Implemented result
Database	MongoDB replica set using six collections
Dataset	137 documents: 20 authors, 40 books, 30 members, 30 loans, 15 reservations and 2 users
Application	Express REST API and responsive staff interface
Security	bcrypt passwords, JWT HTTP-only cookies and role checks
Advanced features	Six aggregations, compound/partial indexes, strict validators and multi-document transactions
Testing	Unit tests, replica-set integration tests, seed verification and desktop/mobile browser checks

Review 2 coverage

This report records the implemented database, the core application functions and evidence for every Review 2 rubric component. Screenshots are included as supporting evidence; the assessment remains centred on database implementation and correctness.

1. System architecture

The system uses a three-layer web architecture. Separating presentation, application logic and persistence makes database operations easier to test and prevents browser code from directly accessing MongoDB.

Layer	Technology	Responsibility	Evidence
Presentation	HTML, CSS, JavaScript	Responsive screens, forms and report tables	Desktop/mobile checks
Application	Node.js and Express	REST routes, rules, authentication and errors	Protected API routes
Data model	Mongoose	Schemas, indexes, references and validation	Six model definitions
Database	MongoDB replica set	Documents, aggregation and transactions	Seed/integration checks
Deployment	Docker Compose	Repeatable application and database services	Start/seed commands

Request flow

Step	Processing
1	The staff member submits a form or opens a report.
2	Express validates the session, role and request body.
3	The route applies business rules and calls the Mongoose model.
4	MongoDB executes an indexed query, aggregation or transaction.
5	The API returns JSON and the interface refreshes the affected view.

2. Database implementation

MongoDB is used because library records contain flexible metadata and benefit from document-oriented storage. Loans and reservations are separate collections because their history grows independently from books and members.

Collection	Key fields	Relationship / purpose	Count
authors	_id, name, country	Referenced by books; avoids repeated author details	20
books	isbn, title, authors, category, copies	Catalog and live stock information	40
members	name, department, email, active	Borrowers referenced by circulation records	30
loans	bookId, memberId, dates, status, fine	Independent issue and return history	30
reservations	bookId, memberId, date, status	Ordered hold queue for unavailable titles	15
users	email, passwordHash, role	Staff authentication and authorization	2

Seed verification result

Authors	Books	Members	Loans	Reservations	Users	Total
20	40	30	30	15	2	137

The deterministic dataset contains available and unavailable books, active and returned loans, overdue records, reservations and two staff roles so that every workflow can be demonstrated.

3. Representative document structures

Document references preserve relationships while keeping transaction history independent. Copy counts are embedded in a book because they are read and updated with the catalog record.

Entity	Example fields	Important rule
Book	_id, ISBN, title, author IDs, category, publisher, copies	ISBN is unique; available cannot exceed total
Member	_id, name, email, phone, department, type, active	Only active members can issue or reserve
Loan	_id, bookId, memberId, issue/due/return dates, status, fine	One active loan per member/book pair
Reservation	_id, bookId, memberId, reservationDate, status	Created only when the title is unavailable
User	email, passwordHash, name, role	Role is limited to admin or librarian

Sample book document

```
{
  "_id": "B001",
  "isbn": "9780001000001",
  "title": "Database System Concepts",
  "authors": ["A001"],
  "category": "Database",
  "copies": { "total": 5, "available": 2 }
}
```

4. Document validation

Mongoose validates application data, while MongoDB JSON Schema validators protect the database when records are inserted outside the web application. Strict validation is enabled for all six collections.

```
const validators = {
  books: { $jsonSchema: {
    bsonType: "object",
    required: ["_id", "isbn", "title", "authors", "category", "copies"]
  }},
  loans: { $jsonSchema: {
    required: ["_id", "bookId", "memberId", "issueDate",
               "dueDate", "status", "fine"],
    properties: { status: { enum: ["Issued", "Returned"] } }
  }}
};
```

Validation rules

- IDs follow collection-specific formats such as B001, M001 and L001.
- Copy counts and fines cannot be negative.
- Loan and reservation status values use fixed enumerations.
- Available copies cannot exceed the total number of copies.
- Email, phone and membership fields are validated before storage.

5. Authentication implementation

The staff portal requires authentication before any collection can be queried or modified. Authentication is separated into explicit stages so that credentials, sessions and authorization are handled consistently.

Stage	Application action	Database/security effect
1. Credential input	Accept email and password over the login route	Input is validated before a database query
2. User lookup	Find the matching email in users	A generic failure message avoids revealing registered accounts
3. Password check	Compare with bcrypt	Only the password hash is stored
4. Session creation	Sign a JSON Web Token	Token is placed in an HTTP-only, same-site cookie
5. Protected request	Run authenticate middleware	Identity is verified before route logic
6. Restricted action	Run role middleware	Admin/librarian permissions are enforced

Security controls

- Passwords are never stored or logged in plain text.
- The browser cannot read the HTTP-only authentication cookie through JavaScript.
- Deletion routes require the administrator role.
- Rate limiting reduces repeated automated login and API attempts.

6. CRUD operations

Operation	API example	Collection	Rule / result
Create	POST /api/books	books	Unique ID and ISBN; validated copy counts
Read	GET /api/books?search=MongoDB	books	Indexed title, ISBN and publisher search
Update	PUT /api/members/:id	members	Valid contact and membership values
Delete	DELETE /api/books/:id	books	Admin-only; protected from unsafe removal
Issue	POST /api/loans	loans + books	Creates loan and decreases stock atomically
Return	PUT /api/loans/:id/return	loans + books + reservations	Fine, stock and queue updated atomically

Example: create and search

```
POST /api/books
{
  "_id": "B041",
  "isbn": "9780001000041",
  "title": "Practical MongoDB Projects",
  "authors": ["A001"],
  "category": "Database",
  "copies": { "total": 3, "available": 3 }
}

GET /api/books?search=MongoDB
```

Business rules

- Only active members can issue or reserve books.
- A book cannot be issued when no copy is available.
- A member cannot hold duplicate active loans for the same book.
- A reservation is created only when the book has no available copy.

7. Application interface: dashboard evidence

After login, the dashboard reads live summary values from the database and presents current circulation activity. The screenshot below is from the seeded and tested application.

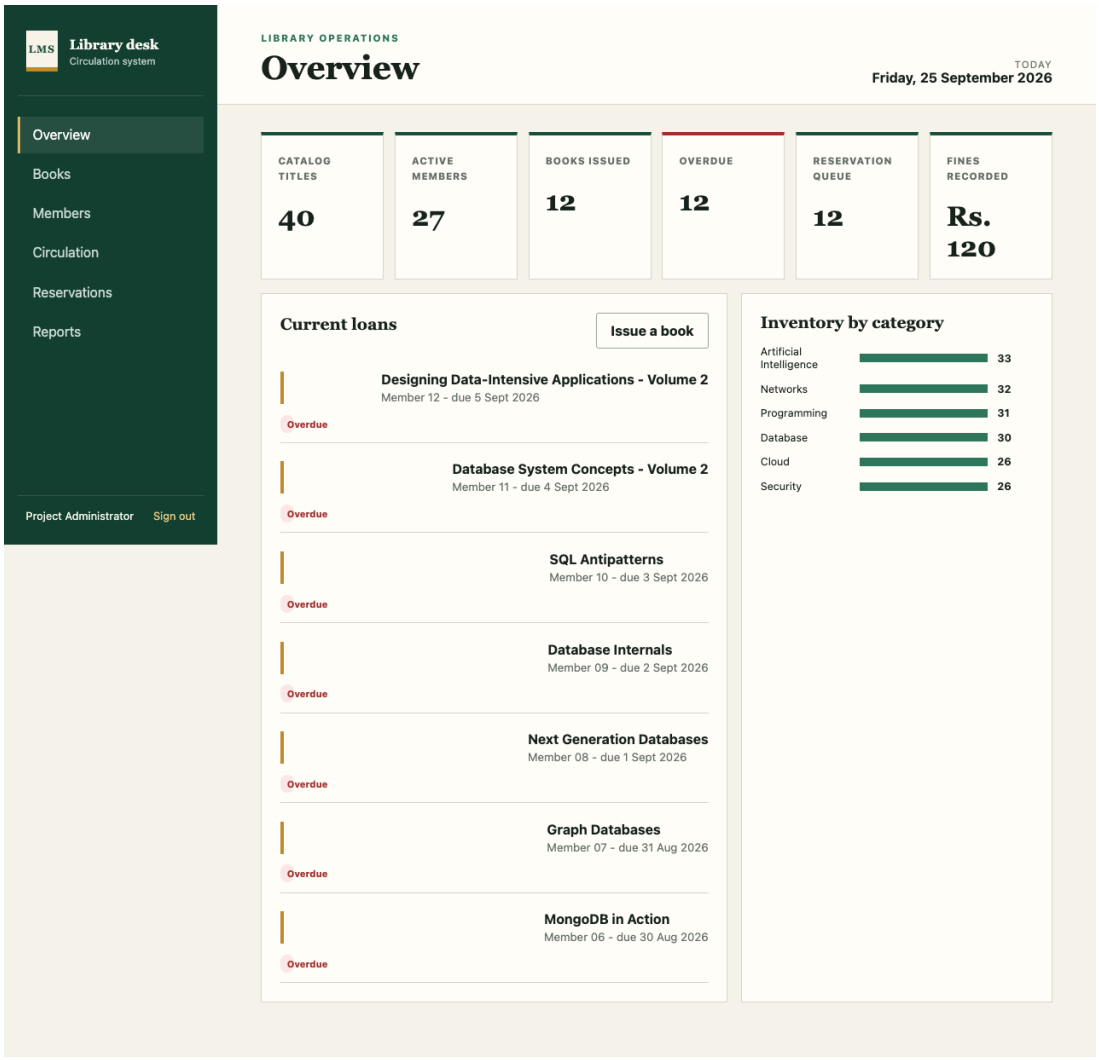


Figure 1. Desktop dashboard showing collection totals, availability and current loans.

Visible evidence	Database source
Catalog title count	Count of documents in books
Issued and overdue values	Filtered loans by status and due date
Available copies chart	Aggregation of embedded copy counts by category
Current loan rows	Loan lookup with book and member details

8. Indexes and authentication

Index	Type	Collection	Purpose
isbn unique	Unique	books	Prevents duplicate catalog entries
book_search	Text	books	Searches title, publisher and ISBN
category_availability	Compound	books	Supports category and stock filters
member_active_loans	Compound	loans	Supports member, active-loan and overdue queries
reservation_queue	Compound	reservations	Finds the oldest active request for a book
one_active_reservation	Unique partial	reservations	Prevents duplicate active holds without blocking history

Authentication controls

Passwords are stored as bcrypt hashes. Login creates a signed JSON Web Token in an HTTP-only, same-site cookie. Middleware verifies the token and checks the admin or librarian role. Helmet security headers and API rate limiting are enabled.

```
app.use(helmet());
app.use(rateLimit({ windowMs: 15 * 60 * 1000, limit: 500 }));
app.use("/api/books", authenticate, booksRouter);
booksRouter.delete("/:id", authorize("admin"), deleteBook);
```

9. Aggregation pipelines

Six reports are generated directly with MongoDB aggregation pipelines.

Report	Important stages	Question answered	Output
Inventory by category	$group, $set, $sort	Where is stock concentrated?	Title and copy totals
Popular books	$group, $sort, $limit, $lookup	Which titles are borrowed most?	Book title and loan count
Overdue loans	$match, $dateDiff, $multiply, $lookup	Who has overdue material?	Member, days and fine
Member activity	$group, $cond, $lookup	How active is each member?	Total/active loans, fines
Monthly circulation	$dateToString, $group	How does activity change monthly?	Loans and returns/month
Reservation queues	$match, $first, $group, $lookup	Who is next for each title?	Queue length, next member

Overdue calculation

```
{ $set: { overdueDays: {
    $dateDiff: { startDate: "$dueDate", endDate: now, unit: "day" }
}}},
{ $set: { calculatedFine: { $multiply: ["$overdueDays", 10] } } }
```

10. Transaction-based circulation

A return changes the loan, book stock and possibly the oldest reservation. These operations run in one MongoDB transaction. If any update fails, all updates roll back.

Step	Return operation	Consistency protected
1	Read the active loan inside the session	Prevents returning a missing/already returned loan
2	Calculate overdue days and Rs. 10/day fine	Uses one return timestamp
3	Set status, return date and fine	Preserves circulation history
4	Increase available copies	Keeps stock synchronized
5	Fulfil the oldest active reservation	Maintains first-come order
6	Commit; otherwise roll back all changes	Avoids partial updates

```
await session.withTransaction(async () => {
  await Loan.updateOne({ _id: loan._id }, {
    $set: { status: "Returned", returnDate, fine }
  }, { session });
  await Book.updateOne({ _id: loan.bookId }, {
    $inc: { "copies.available": 1 }
  }, { session });
});
```

11. Responsive application evidence

The database functions remain usable on a narrow screen. Navigation changes to a menu and data remains readable without removing core circulation information.

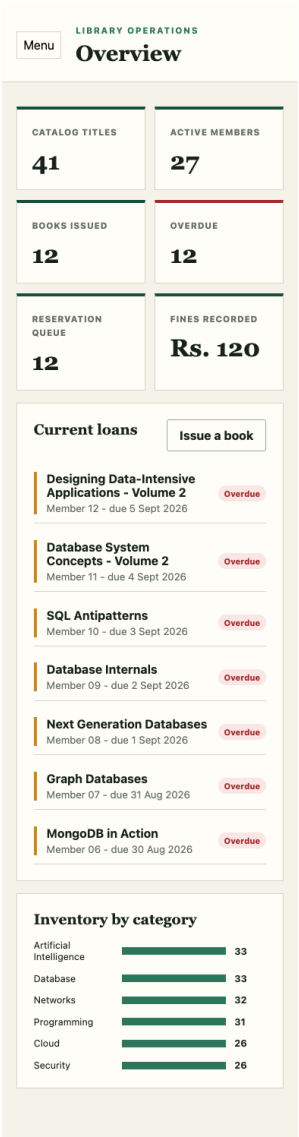


Figure 2. Mobile overview at a 390-pixel viewport.

Responsive behaviour	Verified result
Navigation	Desktop sidebar changes to a mobile menu
Summary cards	Cards wrap vertically without clipped values
Tables	Important columns remain visible; wide data can scroll
Forms	Labels, controls and focus states remain usable
Browser console	No errors were detected during the mobile check

12. Testing and verification

Verification	Method	Expected result	Status
Fine calculation	Four unit tests	Correct zero and overdue fines	Passed
Authentication	Integration request	Protected route accepts valid session	Passed
Issue transaction	Replica-set integration	Loan created and stock reduced together	Passed
Return transaction	Replica-set integration	Loan, fine, stock and queue updated	Passed
Seed counts	Database verification	20/40/30/30/15/2 = 137 documents	Passed
Validators	Collection inspection	Six strict JSON Schema validators	Passed
Desktop UI	Browser workflow	Login, dashboard, books and reports load	Passed
Mobile UI	390-pixel browser check	Navigation and content remain usable	Passed
Console	Browser diagnostics	No runtime errors	Passed

The browser CRUD check created B041 and returned 41 catalog rows. This confirms that a record written through the interface became immediately available to the read/search workflow.

13. Intermediate demonstration plan

Time	Demonstration
0:00	Show six collections, 137 documents, validators and indexes
1:00	Sign in and explain role-based access
1:40	Create, search and update a book; register a member
3:00	Issue and return a book using a transaction
4:30	Create a reservation for an unavailable book
5:20	Show all six aggregation reports and CSV export
6:30	Show responsive interface and source structure

Evidence to keep ready

Evidence	Location / action
Source code	Library_Management_System_Complete folder
Database records	MongoDB collections after npm run seed
Automated verification	Run npm test
Working application	Open http://localhost:3000 after startup
Repeatable environment	Run docker compose up -d --build

14. Review 2 rubric alignment

Criterion	Marks	Implemented evidence	Sections
Database implementation	2	Six collections, 137 records, validators, indexes and repeatable seed	2-4
CRUD operations	3	Books, members, loans and reservations through protected UI/API workflows	5-7
Advanced NoSQL features	5	Six aggregations, text/compound/partial indexes, validation and transactions	8-10
Demonstration quality	Supportin g	Desktop/mobile UI, tests, evidence checklist and reproducible setup	11-13
Total	10	Every stated Review 2 requirement is implemented and evidenced	1-14

Conclusion

The Review 1 design has been implemented as a complete MongoDB application. The database contains representative records, supports the required CRUD workflows and demonstrates multiple advanced NoSQL features with verified output. The accompanying source code and repeatable setup allow each claim in this report to be demonstrated during evaluation.